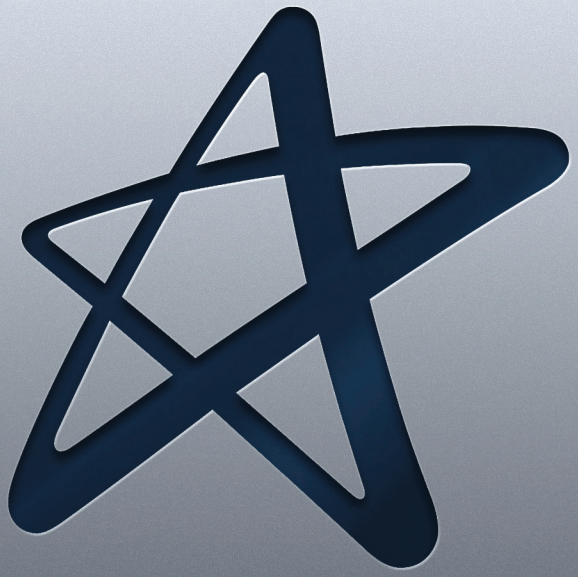
Teoria dos Grafos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Árvores e Ordenação Topológica

Responsável pelo Conteúdo:
Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:
Prof. Me. Luciano Vieira Francisco

UNIDADE Árvores e Ordenação Topológica

- Introdução;
- Árvore;
- Ordenação Topológica e Algoritmos de Fluxo de Rede;
- Fluxo em Redes.



OBJETIVO DE APRENDIZADO

- Conhecer todos os conceitos de árvores e topologia.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta Unidade estudaremos conceitos importantes sobre grafos, árvore, **ordenação topológica e fluxo em redes**. Tais conceitos são de suma importância no estudo de grafos, além de aplicáveis em diversos problemas. Por exemplo, em projeto de circuitos eletrônicos frequentemente é necessário tornar os pinos de vários componentes eletricamente equivalentes, juntando a fiação de todos. Nessa modelagem, para interconectar um conjunto de *n* pinos, podemos utilizar um arranjo de n-1 fios, cada qual conectado a dois pinos. Assim, de todos os arranjos possíveis, aquele que utiliza a quantidade mínima de fios é normalmente o mais desejável. Este problema é facilmente modelado com árvores – veremos mais detalhes.

Ordenação topológica será o segundo assunto estudado nesta Unidade. Podemos dizer que uma ordenação topológica de um grafo pode ser vista como uma ordenação de seus vértices ao longo de uma linha horizontal, de tal forma que todas as arestas orientadas sigam da esquerda para a direita. Ordenação topológica somente é aplicada a grafos orientados e acíclicos. Na teoria de grafos, os acíclicos orientados são empregados em muitas aplicações para indicar a precedência entre eventos.

De uma maneira bem simples, fluxo em rede envolve a complexidade de fazer circular determinado produto através dos vértices e das arestas de uma rede. Habitualmente, os problemas de fluxos são associados às várias situações reais de distribuição de água, eletricidade, produtos industriais, movimentação de veículos, entre outros.

Árvore

De uma maneira bem simples, podemos definir uma **árvore como um grafo conexo e acíclico**. Todavia, essa conceituação é bem sucinta. Uma definição mais completa e tradicional pode se dada a seguir:

Uma árvore é um grafo conexo *T* em que existe **um e somente um caminho entre qualquer par de vértices de T.** Quando árvores são mencionadas, precisamos definir também subárvores. Uma **subárvore** pode ser definida como um **subgrafo conexo e acíclico.**

Outra definição importante também é a de *grafo estrela*: dado um grafo *G* com *n* vértices, um grafo é denominado **estrela** quando *G* é uma árvore que possui **um vértice de grau n-1** e os **demais vértices com grau 1.**

Na teoria dos grafos, árvores são estruturas que podem ser utilizadas para modelar diversos problemas, estes que podem ser encontrados em diversas áreas, desde a computação, até a comunicação. A Figura 1 apresenta alguns exemplos de árvores:

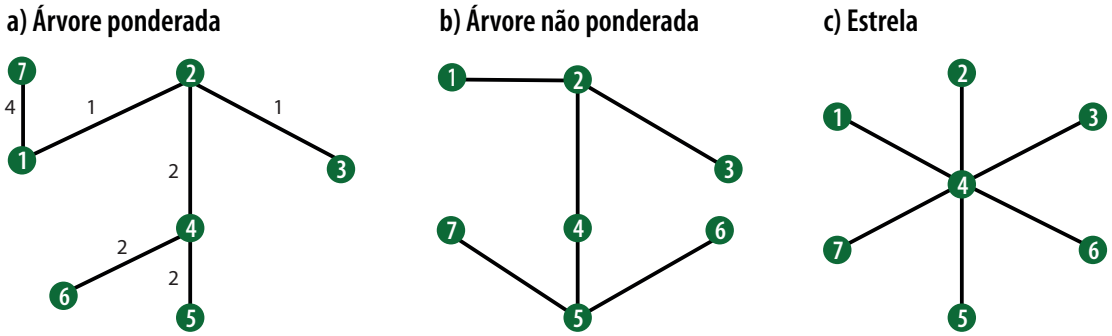


Figura 1 – Exemplos de árvores
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Propriedades das Árvores

A estrutura em árvore possui algumas propriedades básicas:

- Todo grafo *G* conexo possui, pelo menos, uma subárvore que contém todos os seus vértices;
- Toda árvore é um grafo planar;
- Toda árvore finita com $n > 1$ possui, pelo menos, dois vértices terminais.

Árvore Geradora

Uma **árvore geradora** de um grafo *G* é um **subgrafo gerador conexo e acíclico.** Assim, todo **grafo conexo possui, pelo menos, uma árvore geradora.** Vale lembrar que um grafo acíclico é aquele que não possui círculos, ou seja, as suas arestas não formam círculos. A Figura 2 apresenta exemplos de árvores geradas de um grafo:

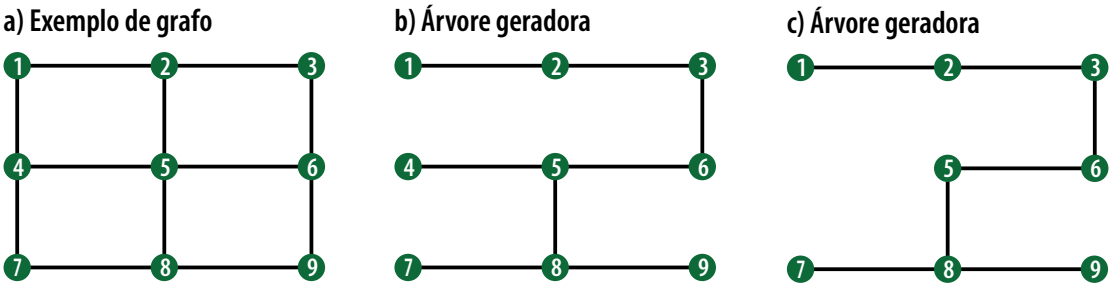


Figura 2 – Exemplo de árvore geradora
Fonte: adaptado de Goldbarg e Goldbarg, 2012

Floresta

Na teoria dos grafos, **floresta** é **um conjunto de árvores sem vértices em comum**. **Uma floresta geradora contém todos os vértices** de *G*. Podemos dizer que uma **floresta geradora é um subgrafo que generaliza o conceito de árvore geradora**. Assim, na floresta geradora cada **componente conexo é uma árvore** e

cada vértice do grafo original está em alguma árvore. De maneira simples, uma floresta geradora em *G* é um subgrafo acíclico e gerador – a Figura 3 exemplifica florestas de um grafo:

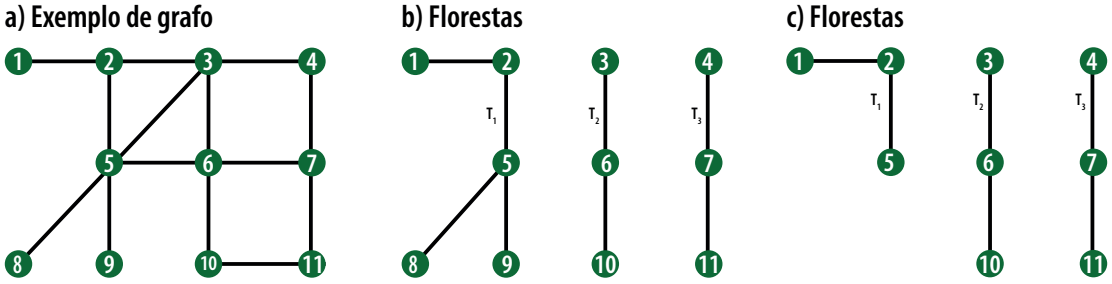


Figura 3 – Florestas
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Aresta Elo

Na teoria de grafos, um conceito importante é o de **aresta elo.** Para entendê-lo considere um grafo G = (N, M) e T (N, M_T), uma árvore geradora de *G*, **uma aresta** $e \in M|M_T$, denominando-se elo *de G em relação a T.*

É importante observar que a adição de um elo em uma árvore produz um único ciclo no grafo resultante – a Figura 4b apresenta uma aresta que produz um ciclo na árvore:

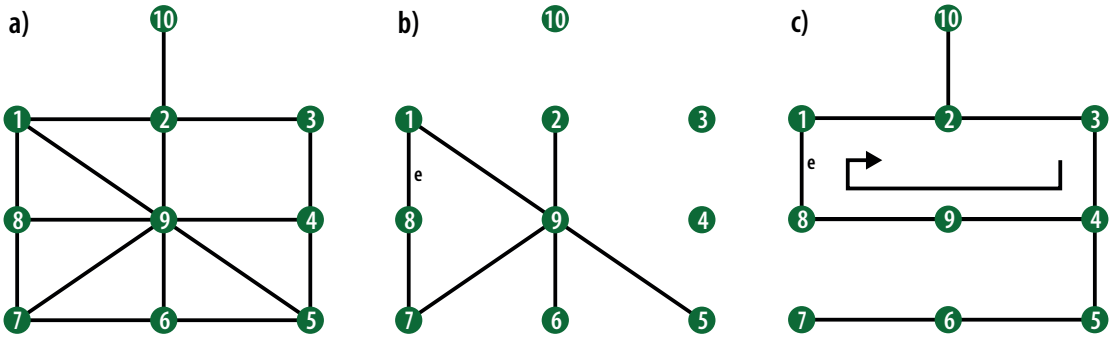


Figura 4 – Ciclo fundamental produzido por aresta elo
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Árvore Geradora Mínima e Árvore Geradora Máxima

Na teoria de grafos, **uma árvore geradora mínima** *(T_{MIN})* é a árvore geradora de **menor custo** entre todas as possíveis em *G*. O **custo** de uma árvore geradora *T* de um **grafo ponderado G** é dado **pelo somatório dos custos das arestas de T**.

Já uma árvore geradora máxima é de **maior** custo em *G*. As figuras 5b e 5c apresentam as árvores geradoras mínima e máxima, respectivamente, do grafo apresentado na Figura 5a:

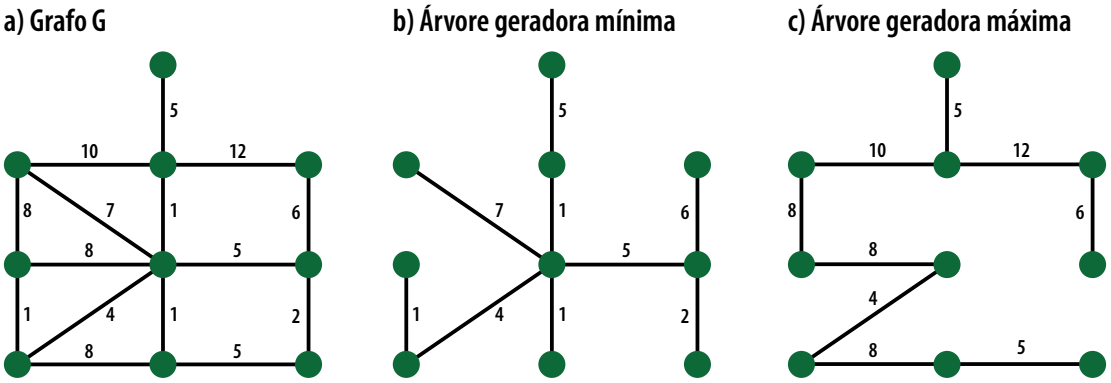


Figura 5 – Árvore geradora mínima e máxima
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Outros dois conceitos importantes são árvore geradora de grau mínimo e árvore geradora mínima com mínimo grau. Começaremos por árvore geradora de grau mínimo, tratando-se de uma árvore geradora desenvolvida em um grafo **não ponderado e possuindo o menor grau máximo possível**. Agora, árvore geradora mínima com mínimo grau é uma árvore geradora mínima de um grafo **ponderado a qual possui o menor grau máximo possível** (GOLDBARG; GOLDBARG, 2012).

A seguir serão apresentados dois algoritmos clássicos para se obter a árvore geradora mínima de um grafo.

Algoritmo de Prim

O algoritmo de Prim é eficiente e elegante. Foi proposto por Robert C. Prim, em 1957. A ideia central desse algoritmo é incluir, de forma gulosa, um a um, os vértices da árvore. São sempre considerados os conjuntos T_{MIN}, T, e V, onde $T_{MIN} \subseteq M$, $T \subseteq N$, $V \subseteq N$. Eis a descrição do algoritmo de Prim (GOLDBARG; GOLDBARG, 2012):

Quadro 1

```
Ler G = (N, M) e D = [d_ij] a matriz de pesos de G
Escolha qualquer vértice i ∈ N
T ← { i }
V ← N \ {i}
T_MIN ← ∅
Enquanto T ≠ N Faça
        Encontrar a aresta (j, k) ∈ M tal que j ∈ T, K ∈ V e d_jk é mínimo.
        T ← T U {K}
        V ← V / {K}
        T_MIN ← TMIN U (j,k)
Fim_enquanto
Escrever T_MIN{o conjunto das arestas da árvore geradora mínima}
```

O algoritmo de Prim parte de qualquer vértice de *G*. A cada passo, acrescenta a menor aresta incidente no conjunto de vértices que já foi selecionado e que possui uma extremidade em vértices fora desse conjunto. A Figura 6 exemplifica o funcionamento do algoritmo de Prim para o grafo apresentado na Figura 6a. A Figura 6e demonstra a árvore final.

Ademais, na Figura 6 é importante observar que a cada passo do algoritmo um novo vértice é acrescentado à árvore em formação através da aresta "mais barata" incidente no conjunto do vértice já examinado. Por exemplo, no passo 3 apresentado na Figura 6c, as arestas examinadas são (2,4), (2,5), (3,4) e (3,5); **o conjunto já examinado é representado pela nuvem pontilhada vermelha.**

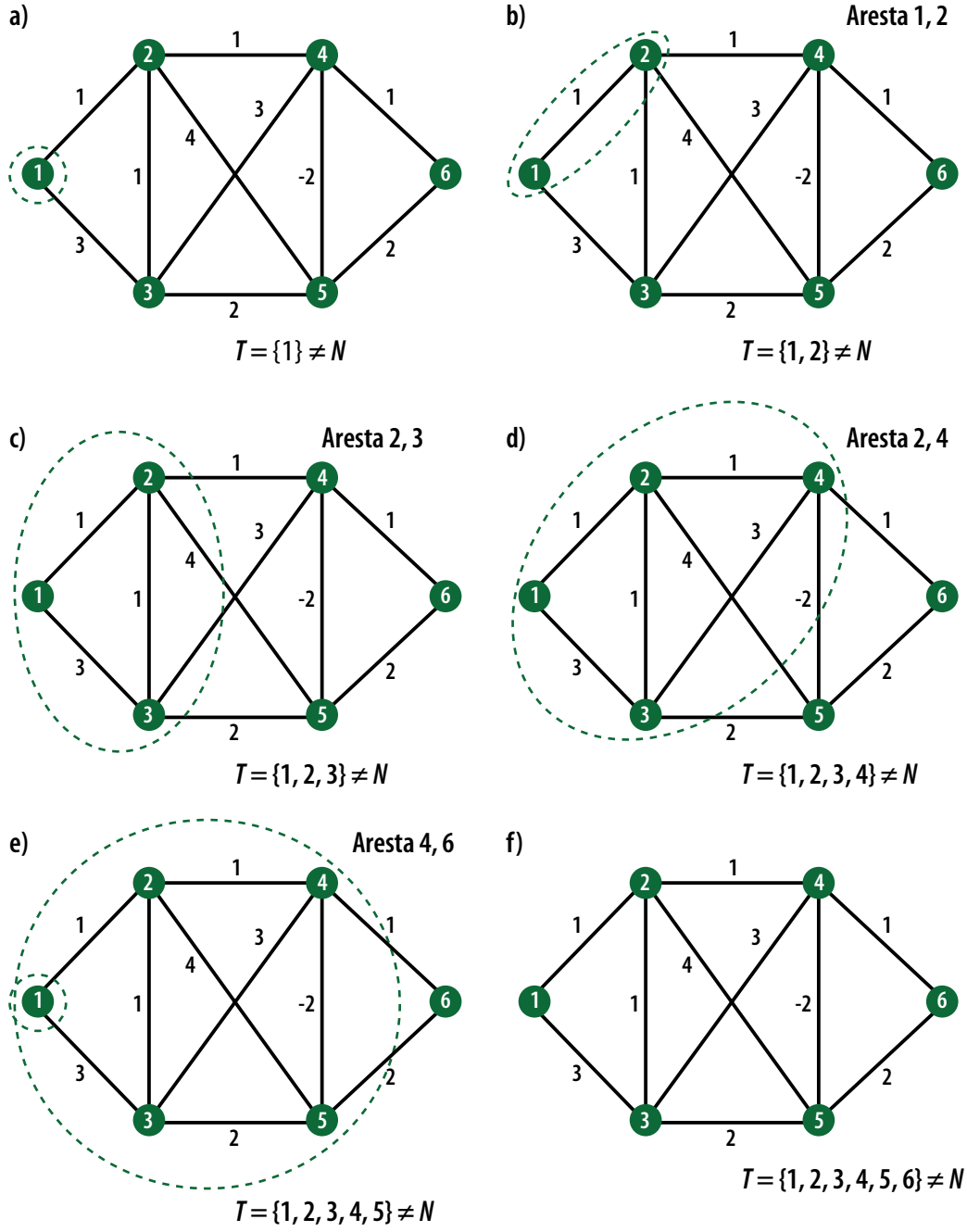


Figura 6 – Evolução do algoritmo de Prim
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Algoritmo de Prim Colorido

O algoritmo de Prim obtém a árvore geradora mínima. Todavia, caso haja um controle das arestas que são examinadas, esse algoritmo ficará mais eficiente. Essa versão do algoritmo de Prim é chamada de **Prim colorido,** vejamos (GOLDBARG; GOLDBARG, 2012).

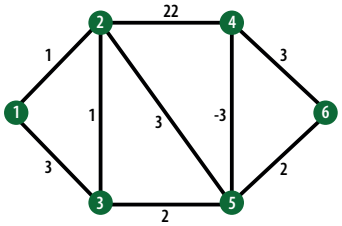
Quadro 2

```
Ler G = (N, M) e D = [d_ij] a matriz de pesos G
Escolha qualquer vértice j ∈ N
I ← 0
Definir T = (N_T, M_T); NT ← {j}; MT ← ∅
Colorir com verde as arestas incidentes do vértice j
Enquanto i < n-1 Faça
        Selecionar a aresta verde (j,k) j ∈ NT, tal que d_jk é mínima e colori-la de Azul
        M_T ← M_T U (j, k); N_T ← N_T U {k}
        Para cada aresta (k, z),  z ∉ N_T Fazer
                Se não existir aresta verde incidente em z, colorir(k, z) como verde
                Senão
                        Se existir aresta verde (w, z) tal que d_wz > d_kz colorir (w,z) com
                        vermelho e (k, z) com verde
        Fim_Para
        i ← i + 1
Fim_enquanto
Escrever T_i {a árvore geradora mínima}
```

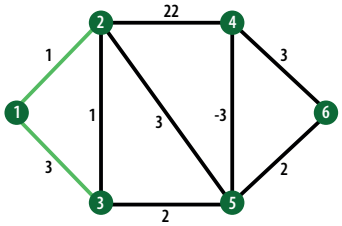
Semelhante ao algoritmo clássico de Prim, o algoritmo de Prim colorido escolhe aleatoriamente o vértice inicial da árvore geradora. O algoritmo de Prim colorido possui complexidade de $o(n^2)$.

Observemos um exemplo: a Figura 7 apresenta o desenvolvimento do algoritmo de Prim colorido. No grafo da Figura 7a o vértice escolhido foi o 1, apresentando o grafo *G*, onde será obtida a árvore gerada mínima; já a Figura 7a (l) ilustra o desenvolvimento do algoritmo de Prim colorido em um passo a passo:

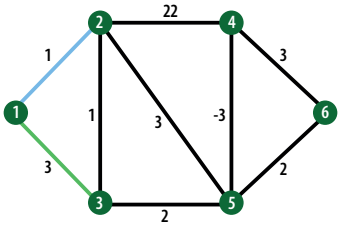
a) Grafo G exemplo para a aplicação do algoritmo de Prim Colorido



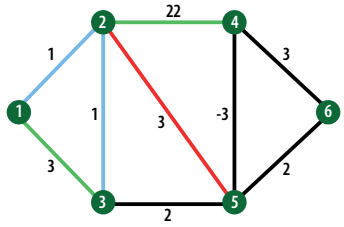
b) Vértice 1 é examinado Arestas (1, 2) e (1, 30) são coloridas de verde



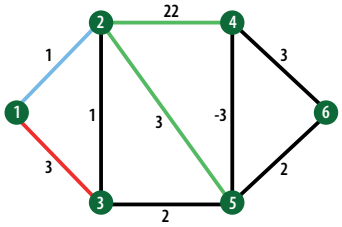
c) A menor aresta verde é colorida de azul e inserida na solução



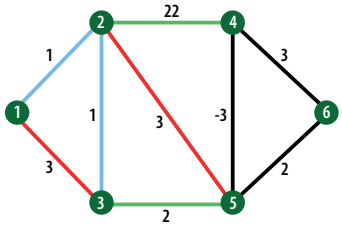
d) Vértice 2 é examinado e as Arestas (2, 4) e (2, 5) são pintadas de verde



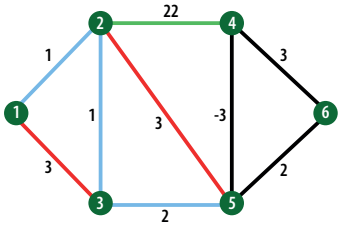
e) Aresta (1, 3) é colorida de vermelho e a Aresta (2, 3) é colorida de azul



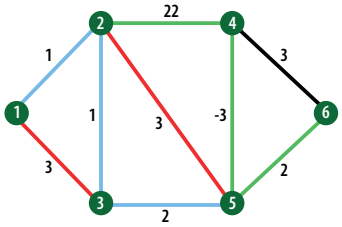
f) Vértice 3 é examinado. A aresta (3, 5) é colorida de verde e a aresta (2, 5) de vermelho



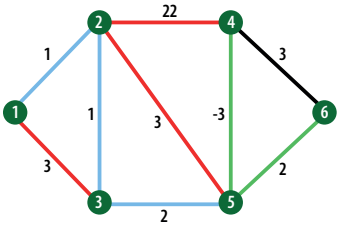
g) Aresta (3, 5) é pintada de azul



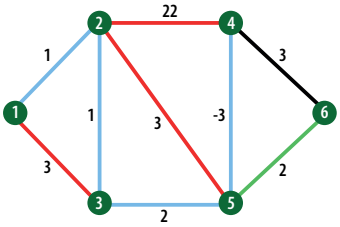
h) Vértice 5 é examinado e as arestas (5, 6) e (5, 4) são coloridas de verde



i) A aresta (2, 4) é pintada de vermelho



j) Dentre todas as verdes, a menor é (5, 4) Então ela é pintada de azul e incluída na solução



l) O vértice 4 é examinado e a aresta (4, 6) é colorida de vermelho e a (5, 6) de azul, i = n –1 e fim

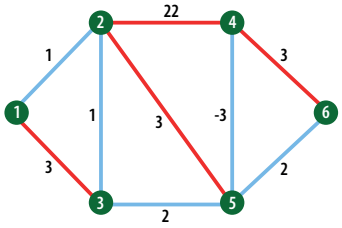


Figura 7 – Desenvolvimento do algoritmo de Prim colorido
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Note que a Figura 7l apresenta a árvore geradora mínima, pintada de azul.

Algoritmo de Kruskal

Além do algoritmo de Prim, diversos outros foram formulados para obter a árvore geradora mínima, entre os quais encontra-se o algoritmo de Kruskal, proposto por Joseph B. Kruskal, em 1956.

A ideia central desse algoritmo é voltada à formação da árvore por meio da inclusão de arestas – e não de vértices, como em Prim. Assim, a "árvore" que se forma é, de fato, uma floresta. A seguir é apresentado o algoritmo de Kruskal (GOLDBARG; GOLDBARG, 2012).

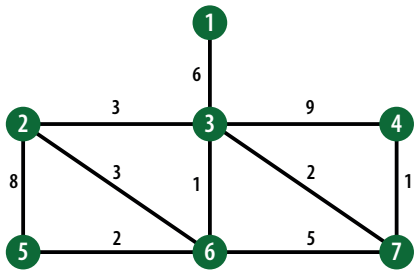
Quadro 3

```
Ler G = (N, M) D = [d_ij] a matriz de pesos de G
Ordene as arestas em ordem não crescente de pesos dij no vetor.
H = [h_i], i = 1, 2, ..., m
T ← h_1
i ← 2
Enquanto |T| < n Faça
        Se T U h_i é um grafo acíclico então
                T ← T U h_i
        i ← i + 1
Fim_Enquanto
Escrever T {arestas da árvore geradora mínima}
```

Nesse algoritmo, *H* representa o vetor das arestas ordenadas e h_j é um elemento de *H*; a árvore em formação é representada por *T*.

As figuras 8a a 8h exemplificam o algoritmo de Kruskal, valendo observar que na Figura 8g a inclusão da aresta (2,6) é recusada porque leva à formação de um ciclo em *T*.

A próxima aresta *H* (6,7) é, então, examinada e incluída na Figura 8h – a próxima etapa do algoritmo Kruskal incluiria a aresta (3,1), concluindo o procedimento.

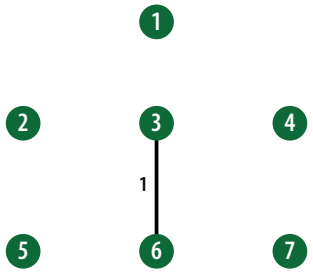


a) Grago G exemplo para o algoritmo Kruskal

Vetor H

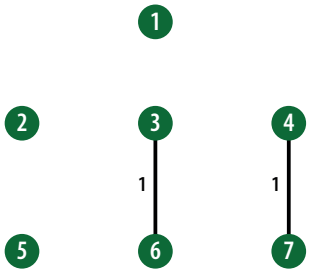
$h_1 - (3, 6) = 1$; $h_2 - (4, 7) = 1$; $h_3 - (5, 6) = 2$;
$h_4 - (2, 3) = 3$; $h_5 - (2, 6) = 3$; $h_6 - (6, 7) = 5$;
$h_7 - (1, 3) = 6$; $h_8 - (2, 5) = 8$; $h_9 - (3, 4) = 9$;

b) Ordenação das arestas de G em H



Vetor H
$h_1 - (3, 6) = 1$;
$h_2 - (4, 7) = 1$;
$h_3 - (5, 6) = 2$;
$h_4 - (2, 3) = 3$;
$h_5 - (2, 6) = 3$;
$h_6 - (6, 7) = 5$;
$h_7 - (1, 3) = 6$;
$h_8 - (2, 5) = 8$;
$h_9 - (3, 4) = 9$;

c) Inserção da Primeira aresta em T



Vetor H
$h_1 - (3, 6) = 1$;
$h_2 - (4, 7) = 1$;
$h_3 - (5, 6) = 2$;
$h_4 - (2, 3) = 3$;
$h_5 - (2, 6) = 3$;
$h_6 - (6, 7) = 5$;
$h_7 - (1, 3) = 6$;
$h_8 - (2, 5) = 8$;
$h_9 - (3, 4) = 9$;

d) Inserção da Segunda aresta em T

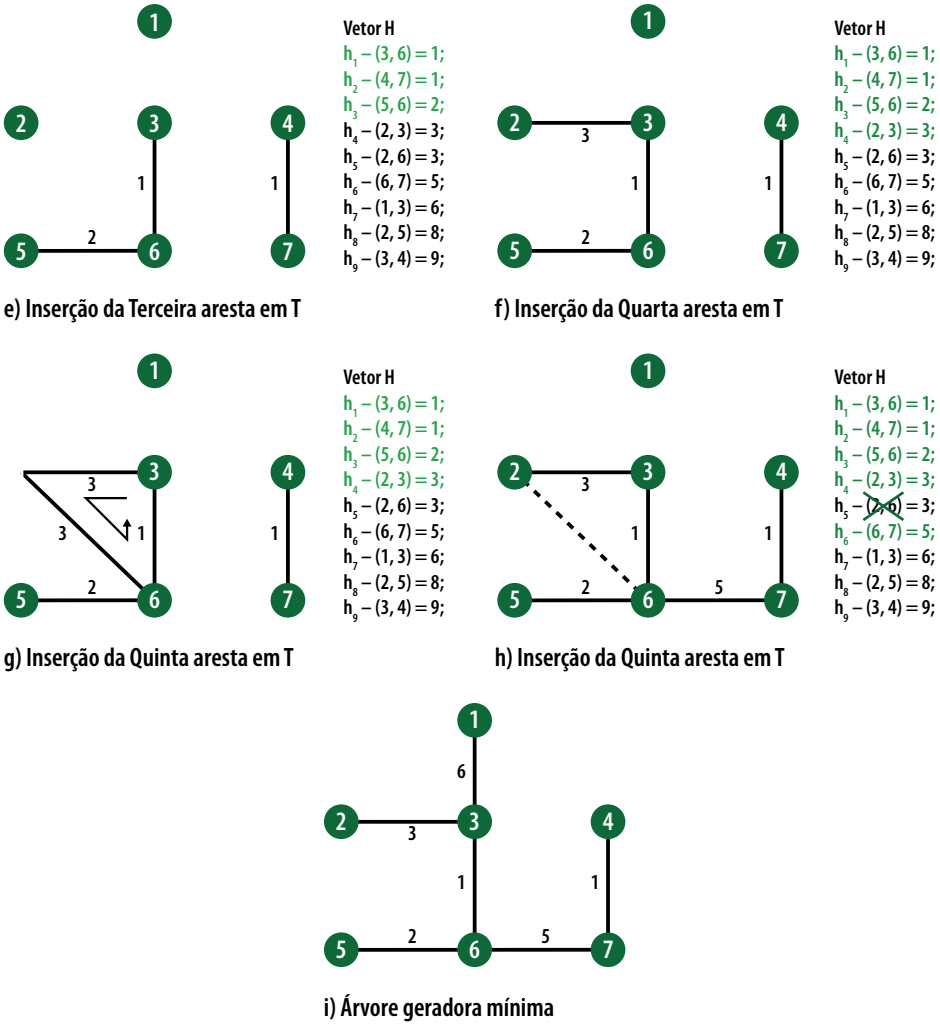


Figura 8 – Desenvolvimento do algoritmo de Kruskal
Fonte: Adaptado de Goldbarg e Goldbarg, 2012

Ordenação Topológica e Algoritmos de Fluxo de Rede

Segundo Cormen e colaboradores (2002), ordenação topológica é uma ordenação linear de todos os seus vértices, tal que se *G* contém uma aresta (u, v), então *u* aparece antes de *v* na ordenação. Para isso, o grafo deve ser acíclico e orientado. Caso o grafo não seja acíclico, então não é possível nenhuma ordenação linear.

Podemos dizer que uma ordenação topológica de um grafo pode ser vista como uma ordenação de seus vértices ao longo de uma linha horizontal, de tal forma que todas as arestas orientadas sigam da esquerda para a direita.

Na teoria de grafos, grafos acíclicos orientados são empregados em diversas aplicações para indicar a precedência entre eventos. A Figura 9 mostra um exemplo que surge quando o hipotético **professor João** se veste pela manhã: deve trajar certas

peças de roupas antes de outras – por exemplo, meias antes de sapatos –; já outros itens podem ser colocados em qualquer ordem. Assim, no grafo apresentado na Figura 9 uma aresta *(u, v)* indica que a peça de roupa *u* deve ser vestida antes da peça *v*.

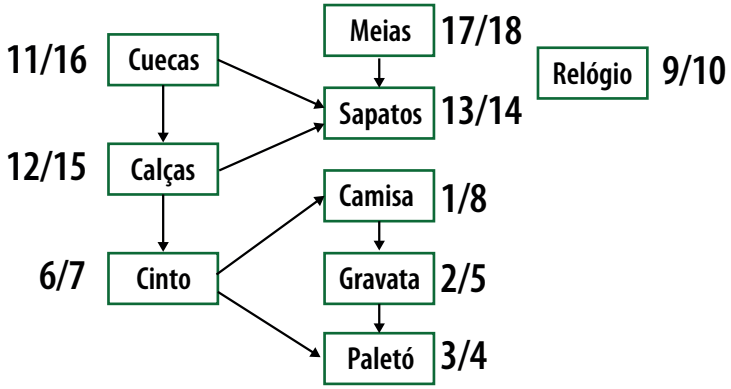


Figura 9 – Grafo do exemplo
Fonte: Adapatado de Cormen e colaboradores, 2002

Portanto, uma ordenação topológica desse grafo fornece uma ordem ao processo de se vestir (CORMEN *et al.*, 2002).

A Figura 10 apresenta o grafo topologicamente organizado como uma ordenação de vértice ao longo de uma linha horizontal, tal que todas as arestas orientadas sigam da esquerda para a direita:

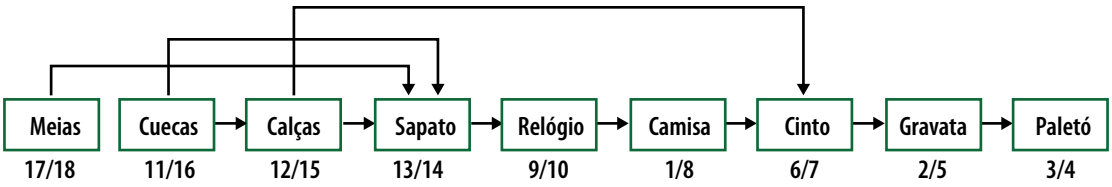


Figura 10 – Ordenação topológica
Fonte: Adaptado de Cormen e colaboradores, 2002

Fluxo em Redes

Segundo Goldbarg e Goldbarg (2012), os problemas de fluxo abordam a complexidade de fazer circular determinado produto através dos vértices e das arestas de uma rede. Assim, esse tipo de problema associa aos componentes de um grafo já conhecido um novo elemento denominado **fluxo**. Habitualmente, os problemas de fluxo são associados às diversas situações reais de distribuição de água, eletricidade, produtos industriais, movimentação de veículos, entre outros fatores.

Em fluxo de redes, a distribuição dos produtos não é realizada obrigatoriamente de um ponto de produção a um ponto de demanda, assim, é possível que existam pontos intermediários, tais como depósitos ou centros de concentração e distribuição. Todavia, no trajeto do objeto pela rede pode haver algumas restrições, tais como de capacidade de tráfego, de custos etc.

Em fluxo de rede, sempre é procurado manter o maior fluxo possível, mesmo havendo restrições – de custo ou capacidade, por exemplo –, isso é conhecido como o **problema do fluxo máximo**. Na literatura foram propostos diversos algoritmos a esse tipo de problema, tais como Dantzig, Ford & Fulkerson, Edmonds & Karp, entre outros.

Em Síntese

Nesta Unidade estudamos conceitos importantes sobre grafos, tais como árvore, **ordenação topológica** e **fluxo em redes**. Esses conceitos são de suma importância no estudo de grafos, sendo aplicados em diversos problemas. De uma maneira bem simples, podemos definir **árvore como um grafo conexo e acíclico**.

Ordenação topológica foi o segundo assunto estudado nesta oportunidade, de modo que podemos dizer que uma ordenação topológica de um grafo pode ser vista como uma ordenação de seus vértices ao longo de uma linha horizontal, de tal forma que todas as arestas orientadas sigam da esquerda para a direita. Ademais, **fluxo em rede** envolve a complexidade de fazer circular determinado produto através dos vértices e das arestas de uma rede.

Por sua vez, árvore geradora de um grafo *G* é um subgrafo gerador conexo e acíclico.

Logo, na teoria de grafos uma árvore geradora mínima (T_{MIN}) é a árvore geradora de menor custo entre todas as possíveis em *G*. Existem dois algoritmos clássicos para se obter a árvore geradora mínima, os algoritmos de Kruskal e de Prim.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à teoria dos grafos

CLÁUDIO, L. L. **Introdução à teoria dos grafos**. [S.l.]: Impar, 2016.

Fundamentos da teoria dos grafos para computação

NICOLETTI, A. M.; HRUSCHKA JR, E. R. **Fundamentos da teoria dos grafos para computação**. São Carlos, SP: Edufscar, 2006.

Grafos e redes: teoria e algoritmos básicos

SIMÕES, J. M. S. **Grafos e redes**: teoria e algoritmos básicos. [S.l.]: Interciência, 2013.

Leitura

Matemática discreta: combinatória, teoria dos grafos e algoritmos

http://bit.ly/2lvLc8r

Referências

BOAVENTURA NETTO, P. O. **Grafos**: teoria, modelos, algoritmos. 2. ed. São Paulo: Blucher, 2001.

_____.; JURKIEWICZ, S. **Grafos**: introdução e prática. 2. ed. [S.l.]: Blucher, 2017.

CORMEN, T. *et al.* **Algoritmos** – teoria e prática. 2. ed. [S.l.]: Campus, 2002.

FEOFILOFF, P.; KOHAYAKAWA, Y.; WAKABAYASHI, Y. **Uma introdução sucinta à teoria dos grafos**. 2011. Disponível em: <http://www.ime.usp.br/~pf/teoriadosgrafos>. Acesso em: 24 nov. 2018.

FURTADO, A. L. **Teoria dos grafos algoritmos**. Rio de Janeiro: LTC, 1973.

GOLDBARG, M.; GOLDBARG, E. **Grafos** – conceitos, algoritmos e aplicações. [S.l.]: Campus, 2012.

NICOLETTI, A. M.; HRUSCHKA JR, E. R. **Fundamentos da teoria dos grafos para computação**. São Carlos, SP: Edufscar, 2006.

SIMÕES, J. M. S. **Grafos e redes**: teoria e algoritmos básicos. [S.l.]: Interciência, 2013.

SZWARCFITER, J. L. **Teoria computacional de grafos**. [S.l.]: Elsevier, 2018.


Cruzeiro do Sul
Educacional